

0 我的高频易错(先看)

- 方法调用带 () ;List用get(i)非 [i] ;递归返回List用addAll。
- 条件 == 非 != ;英文引号; import java.util.*; (空格非点)。
- 非void别忘 return;void不能赋值 (l=Collections.sort())X)。
- 读文件 New Scanner(new File(f)) 非 new Scanner(f)。
- 方法不能套方法;main必须在class里;类体内才能写构造器。
- 循环求和用累加器,别在for里return;解析后要add(new 对象 (...))。
- 构造器赋值别加类型:Map<, > p=new...=局部变量遮蔽,字段null。
- throw new X() 带括号;getX()+=vX→put(id,get(id)+v)。
- 集合字段必须new初始化;1/2=0 (面积用b*h/2)。
- T只能extends? super读出=Object;double用Double.compare。
- lambda/匿名类赋值末尾; ;checked异常放try里;枚举abstract末尾要声明。

1 编译运行·程序结构·考试

```
javac *.java && java Pet 20 Rex //java跟类名,无.class/.java
int age=Integer.parseInt(args[0]); //args全是String
double w=Double.parseDouble(args[1]); //个数args.length非size()

• 文件名=public类名;1个.java仅1个public类;main入口。作用域由{}决定;内层见外层,反之不行,出域用=编译错。

! java Pet.classX javac Pet,VehicleX;OK:javac *.java; java Pet
```

2 类型·整除·char·装箱·运算

- primitive不能=null;String/数组=引用类型;字段默认0/false/null,局部变量无默认值。

```
9/2=4 (double)9/2=4.5 (double)(9/2)=4.0
1/2.0=0.5
10/3+10%3=4 (int)3.9=3 1/2=0! 7%3=1 -7%3=-1
5/0=ArithmeticException(Runtime) 5.0/0=Infinity
0.0/0.0=NaN
'A'+1=66 (char)('A'+1)='B' 'A'+ 'B'=131
'A'+ "BC"→"ABC"
'5'-'0'=5 i=5;print(i++)=5(后6) print(++i)=7
Integer a=127,b=127:a==b T(缓存-128~127);200=F(新对象)-比值必用equals!
MAX_VALUE+1=绕回MIN Integer i=null;int j=i;-NPE拆箱
Integer.parseInt("42")=int基本型
Integer.valueOf("42")/valueOf(42)=Integer对象(用缓存)
String.valueOf(42/true/'A')→"42"/"true"/"A" //任意-String

! 短路:true||boom()跳过,&不短路;优先级 %/> +>- 比较 > && >|| >?:>=。强转位置:(double)(x/2) 先除。
```

```
printf:第2整%I 小数6位)%S串%c字符%b布尔%n换行%;%.2f留2位)%5d右%-5d左%05d补零;不自动换行。三元a>b?a:b是表达式可返回。
```

2b 控制流·switch

- do{...}while(c);至少执行1次;for-each只读元素无下标、不能遍历无边增删、要控制用普通for;标签break:outer: for(...) {for(...)break outer};
- switch支持int/char/String/enum;case无break→fall-through落空到下一case;多case共享;enum case不加类名前缀;default任意位置。

3 类·构造器·static·final

```
public class Person{
    private String name; private static int count=0; //全类一份
    public Person() { this("Jeff",21); } //this()调本类构造器,首行
    public Person(String n,int a){ this.name=n; count++; }
    public static int getCount(){ return count; } //static禁this
}
```

- 构造器与类同名无返回类型(public void Car()){普通方法! 考过);写带参后默认无参消失。this.x区分同名参数/字段(遮蔽shadowing:同名时裸名指参数)。
- static属类:类名调、全类共享、无this不能直接用实例成员(要对象);实例方法可用static。obj.静态()能编译但按声明类型、obj=null也不炸(不推荐)。static调static:同类直接调,跨类类名调;static里没有super/this→调父类static用父类名.方法() (super只用于实例方法调被重写版本)。何时static:不依赖对象状态(Math.max);何时实例:读对象状态(getBalance)。
- final变量=常量/方法禁重写/类禁继承;接口字段隐含public static final。
- 三种变量:instance field实例字段(无static,属对象,obj.x,每对象一份)/static field类字段(有static,属类,类名.x,全类共享一份)/local variable局部变量(方法内声明,方法结束即销毁,用前必须先赋值)。判断:字段前没写static=实例字段。

* 构造器给字段赋值别加类型(加=新局部变量遮蔽字段,字段null→NPE;固定值声明处或构造器赋均可,参数值只能构造器this.x=x;集合字段必须初始化new ArrayList<>())。

4 访问修饰符·package·import·UML

	pub	prot	默认	priv
本类	Y	Y	Y	Y
同包	Y	Y	Y	N
异包子类	Y	Y	N	N
异包非子类	Y	N	N	N

- package首行对应目录;异包必须import,同包/java.lang(String/Math/System/Integer)不用。java.util.*整包/.Scanner单类/import static org.junit.Assert.*静态导入免类名。

```
//常用具体import(教材就用具体类,非*) :
import java.util.Scanner; import java.util.List;
import java.util.ArrayList; import
java.util.LinkedList;
import java.util.Map; import java.util.HashMap;
import java.util.TreeMap; import java.util.Set;
import java.util.HashSet;
import java.util.Arrays; import
java.util.Collections; import
java.util.Comparator;
import java.io.File; import
java.io.FileNotFoundException; import
java.io.PrintWriter; import java.io.FileWriter;
import java.io.IOException;
import org.junit.Test; import org.junit.Before;
import static org.junit.Assert.*;
```

```
//zoo/Animal.java
package zoo;
public class Animal{ }
//app/Main.java
package app;
import zoo.Animal; //跨包=必须import
public class Main{ Animal a=new Animal(); }

• UML: +pub -priv #prot;斜体=abstract;interface;实线三角=extends(Generalization),虚线三角=implements(Realization);泛型类右上角虚线框T;name:String / m():void。
```

5 继承extends·super·创建顺序

```
class GlassBottle extends Bottle{
    public GlassBottle(){ super(10,10,10); } //须首行
@Override public double volume() { return broken?
0:super.volume(); }
}
```

- 单继承;隐式继承Object,private不被继承(用getter),protected子类可用;父类不知子类。is-a才继承,has-a组合(对象作字段:Car{Engine e};)。
- 构造链先父后子:不写super→隐式super();父类只有带参→子类必须显式super(args) 否则"constructor cannot be applied";父类有无穷(或都有)→可省略自选。
- super(...)/this(...)都须首行不能同时;super(...) +this. 字段=.. 共存OK(子类字段super后赋)。super.m() 调父类版本(任意位置);字段同名遮蔽用super.x(少用)。
- new 子类完整顺序:1.static块(一次,父→子) 2.父字段初始化 3.父构造器体 4.子字段初始化 5.子构造器体→输出题父类构造器先打印。

! Sub s=new Super() X。接口继承接口无super(无构造器,抽象方法没body可调);要调父接口default: A.super.m() (罕)。

6 重载Overload vs 重写Override

	重载	重写
参数	不同	相同
位置	同类	子类
绑定	编译期	运行期
返回	任意	相同/协变

- X只改返回类型 X只改参数名 X只改修饰符;@Override查签名(拼错/改参=报错);重写不能降低可见性、不能抛更宽checked;static方法=隐藏(按声明型)非重写;构造器不继承可重载(this...链)。父子同签名:实例→实例=重写✓/static→static=隐藏✓/static与实例互盖=编译错。
- 匹配:精确>提升(int→double)≥装箱 (int→Integer):deduce(1,2)→(int,int);(2,2.0)→(int,double);((double)3,2)→(double,double);("42",123)→(String,int) 优于Integer;双null歧义→(int[])null 强转。

7 多态·转型·equals/hashCode·字段隐藏

能调啥看声明类型(编译),执行谁看对象(运行):Animal a=new Dog() OK上转;A.bark() X;子类特有)→((Dog)a).bark();Dog d=new Animal() X;转型强转→运行ClassCastException,先instanceof;兄弟类不能互转。

== 引用; .equals() 内容;Object默认equals即==。重写equals参数必须Object:this==o 判到 ! (o instanceof X) - 强转比字段,必同时重写hashCode(Objects.hash)否则HashMap/Set查不到。

```
class A{int x=1;int f(){return x;}} class B
extends A{int x=2;int f(){return x;}}
A a=new B(); a.x=1(字段=声明型!) a.f()=2(方法=对象)((B)a).x=2
```

! 没重写ToString=类名@1be6f5c3;println(对象)/拼接自动调toString; "+null"→"null"安全,null.m()→NPE。多态例子:println(Object)/Scanner构造重载/catch(Exception)。

8 抽象类 vs 接口·default·static

	abstract类	接口
字段	任意	仅public static final
构造器	有(供super)	无
方法	具体+abstract(可0个)	隐式public abstract+default+static
继承	extends 1	implements多;接口extends多接口OK

- 都不能new(匿名类除外);抽象方法无方法体仅abstract类/接口;具体子类必须全实现否则也abstract;extends父implements接口,接口(extends在前)。
- default=接口方法有实现(考过);两接口同名default→实现类必须重写;接口static方法不被继承:A默认打A+B静态打B,C实现两者→new C().m()→A。
- 接口static必须有方法体;只能 接口名.方法() 调(实现类名X 对象X;类的static可经子类名/对象调但不推荐——接口static最严)。
- 菱形问题=多继承下同名方法两份实现不知用哪个(类单继承堵死)。两接口同名签default→编译错,实现类必须重写;体内随意(空新逻辑/A.super.m()借用某边)——类实现→接口default,重写=替换两份全盖掉,不是替你选;参数不同=签名不同→无冲突,两个都继承(自然重载);正常单接口default可选不重写;default+static同名不冲突(static不被继承)。

- 接口当类型:Move m=new Dog();参数 sort(list,Comparator c) 可传实现类/匿名类/lambda;写接口名不能写关键字interface;接口可作返回类型/字段类型。

9 String不可变·池·方法·StringBuilder

```
String a="Alice",b="Alice",c=new String("Alice");
a==b=T(池中同对象) a==c=F(new必新建) a.equals(c)=T
String s="String is fun"; //下标S0 t1 r2 i3 n4 g5
s.length() -12 //长度(含空格)
s.charAt(0) -'S' //第1个字符
s.indexOf('i') -3 //首次出现下标,无-1
s.indexOf("is") -7 //子串也行
s.lastIndexOf(" ") -9 //最后一个空格
s.contains("fun") -true //含不含子串(区分大小写,传String)
s.substring(0,6) -"String" //[0,6) 含头不含尾
s.substring(7) -"is fun" //到结尾
s.split(" ") -{String,is,fun} //按分隔切成String[]
s.equals("String is fun") -true //比内容(==比引用别用)
s.equalsIgnoreCase("STRING IS FUN") -true
s.compareTo("Apple") -正 //字典序,负=前 0=同 正=后
s.toUpperCase() -"STRING IS FUN"
s.toLowerCase() s.trim()去首尾空格
s.replace('i','X')
s.startsWith("Str")-T s.endsWith("un")-T
s.isEmpty()-F s.toCharArray()

• 不可变:每个方法都返回新串,原串不动→必须s=s.xxx() 接住,调了不赋值=白调。

//StringBuilder:可变,循环拼接快(String每次+都新建对象慢)
StringBuilder sb=new StringBuilder(); //或 new
StringBuilder("abc")
sb.append("Hi"); //追加,可
链:sb.append(x).append(y)
sb.append(5).append('!'); //任意类型都能append
sb.insert(2," "); //在下标2处插入
sb.delete(0,2); //删[0,2)
sb.deleteCharAt(0); //删一个
sb.replace(0,2,"XY"); //替换[0,2)
sb.reverse(); //反转
sb.charAt(0) sb.length() sb.setCharAt(0,'Z');
String result=sb.toString(); //最后转回String才能用/打印
```

- reverse()只StringBuilder有,String没有!反转字符串:new StringBuilder(s).reverse().toString();反转List用Collections.reverse(list)。
- 末字符s.charAt(s.length()-1);长度三兄弟:数组.length 属性/String.length()方法/集合.size()方法。

10 值传递·打印数组·二维

- Java只有值传递,拿到的永远是副本:基本类型=值的副本;对象/数组=引用的副本(对象本身不复制,两副本指同一对象)。方法内a=new数组只调局部引用外部不变;a[0]=x改内容外部可见(Q12:update后[Python, Java]没变,mutate后[C#, Java]);返回值没人接收(update(s);)=直接扔掉。
- 打印:数组Arrays.toString(arr) (返回String自带[]);嵌套deepToString(递归展开);直接println数组→[I@1be6...List/Map直接println即可;Arrays.sort(arr) 排序。
- 二维=数组的数组:new int[3][4] 全0;a.length行,ai[].length列;jagged:new int[2][] 后每行单独new;遍历for(int[] row:a)for(int v:row)。

11 异常:分类·try/catch·throw/throws

ThrowableError(别catch)=StackOverflow(无base递归)/OutOfMemory/Assertion
↳Exception checked(编译强制):IO/FileNotFound(IO子)/ClassNotFoundException/Interrupted/SQLError/runtimeException unchecked:NullPointerException/ArrayIndexOutOfBoundsException/OutOfMemoryError/Arithmetic(int=0)/NumberFormatException(parse失败,IllegalArg)/IllegalArgumentException/IllegalState/ClassCast/NegativeArraySize

- 编译错(类型/分号/找不到)≠异常对象,运行期才抛。checked:catch或throws二选一(每方法各管;抛而不catch→该方法必须throws;调用者再选;文件习惯try-catch,业务校验习惯throws给上层);unchecked不强制无需throws。
- throw=体内抛实例带括号 throw new X();throws=签名声明,多个逗号隔开。谁抛:库/JVM自动(文件/NPE/越界/除0)你只catch;自己的规则自己throw(校验/自定义),否则永不触发。
- 多catch子类在前(反—"already been caught"编译错);finally必执行(即使return);try中抛点后代码跳过直接catch;被catch后程序继续。throw后立即退方法;if无return/else→后面两种情况都跑(打印错误后return); catch(Exception)接不到Error。

```
class InsufficientItemException extends
Exception{ //自定义checked
    public InsufficientItemException(int id){ //构造器在类体内!
        super("Insufficient number of items for
"+id); } } //super在构造器里
void sell(int id,int n) throws
InsufficientItemException{
    if(!products.containsKey(id))
    { System.out.println("no id"); return; }
    int amt=products.get(id);
    if(n>amt) throw new
InsufficientItemException(id);
    else products.replace(id,amt-n); }
```

* Xclass X extends Exception(参数) 参数不能接类名后!
另一式:构造器收String X(String m){super(m); }→throw new X("...") 两种都对。extends
RuntimeException→unchecked无需throws。违反precondition→throw new
IllegalArgumentException("...") (unchecked可不catch);
内置异常String消息可选。测两种异常用两个独立try(同try第一个抛后第二个不执行)。

12 文件I/O- GiftCard模板

```
try{ Scanner sc=new Scanner(new File(name)); //名
    可来自变量/args/键盘输入
    while(sc.hasNextLine()){ String
l=sc.nextLine(); ... }
    sc.close();
}catch(FileNotFoundException e)
{ System.out.println("not found"); }
try{ PrintWriter w=new PrintWriter(name); //覆盖写;
    追加new PrintWriter(new FileWriter(f,true))
    w.println(x); w.close(); //close必须否则丢数据
}catch(FileNotFoundException e){ }
```

- 两者抛checked FileNotFound → try/throws; Scanner(System.in) 不抛(构造器重载); token流 hasNext()/next()/nextInt()。
- 带空格行: i=line.lastIndexOf(" "); 名=substring(0,i); 数=parseInt(substring(i+1))。!nextInt后接nextLine先sc.nextLine()吃换行。套路:读→解析→add(新 对象)→return list(读文件建议独立方法返回List,main串流程)。

```
public class GiftCard{ //Q17原题
    private double amount;
    public GiftCard(double a){ amount=a; }
    public void buyBook(double p){
        if(p<=amount){ amount-=p;
        System.out.println("Books bought successfully"); }
        else System.out.println("Not enough money on
the card"); }
    public double amountRemaining(){ return
amount; } }
```

* 模板:命令行+对象数组(Review真题)

```
class Employee{ String name; double salary;
    Employee(String n,double s){ name=n;
salary=s; } }
public class Main{ public static void
main(String[] args){
    Employee[] es=new Employee[args.length/2];
    int num=0; String name=null; double max=0;
    for(int i=0;i<args.length;i++){
        if(i%2==0) name=args[i];
        else{ double s=Double.parseDouble(args[i]);
        es[num++]=new Employee(name,s);
        if(s>max) max=s; } }
    for(int i=0;i<num;i++){
        if(es[i].salary==max)
        System.out.println(es[i].name);
    } }
```

13 集合- 自写链表

- 泛型只装引用类型(List<int>× Integer自动装箱); ArrayList下标O(1)快(头中插删慢(搬移)); LinkedList节点散布内存(考过T)两端快addFirst/Last,removeFirst,getFirst; HashMap无序,put重复覆盖,get无→null,getDefault(k,0),containsKey,replace; TreeMap按key排序(key非null); Set无重复add重复无效,TreeSet有序。
- 遍历Map: for (Map.Entry<K,V> e:m.entrySet()) getKey/getValue;或keySet后get。
- List.of(1,2,3) → 一行建列表但完全不可变: get/size/遍历✓, add/remove/set → UnsupportedOperationException; 要可改套new ArrayList<>(List.of(1,2,3)) 拷成可变。Arrays.asList(a,b) 固定长(set可, add/removeX); 传int[] 当1个元素(用Integer)。

! for-each中
list.remove() → ConcurrentModificationException(改用普通for或先收集再删); 自定义类作key/放Set → 重写equals+hashCode。

```
class MyList<T>{ //自写泛型链表(常考)
    private class Node{ T value; Node next; Node(T v)
    {this.v=v; } } //内部类可访问外层
    private Node head,tail; private int size;
    void add(T v){ Node n=new Node(v);
        if(head==null){ head=tail=n; }
        else{ tail.next=n; tail=n; } size++; }
    T removeFirst(){ if(head==null) return null;
    T v=head.v; head=head.next;
    if(head==null) tail=null; size--; return v; }
    int count(){ int c=0;
        for(Node cur=head; cur!=null; cur=cur.next) c++;
        return c; } } //!cur.next!=null少1(Review考过);
游标必须cur=cur.next赋值才动
```

删index: 停在index-1, cur.setNext(cur.getNext().getNext()); 删头head=head.next特殊处理。

14 泛型-边界-限制-Math

- class Box<T> (构造器名不带<T>; 多参Pair<K,V>; new菱形<>; 泛型方法<T>在返回类型前: static <T> find(T n,T[] h); 调用用Points.<P>find(..)或推断。
- 边界只能extends(类/接口同): OK<T extends Number>(可调doubleValue/上界方法) X super X implements X bound(考过); <T extends Comparable<T>> → T有compareTo(负0正)。

```
//泛型求最大(Q30: 边界保证能compareTo; 数组版用T[],xs[0])
static <T extends Comparable<T>> T max(List<T> xs)
{
    T m = xs.get(0);
    for(T x : xs) if(x.compareTo(m)>0) m = x;
    return m; }
```

- 类型擦除: 运行期<T> → Object → Xnew T() Xnew T[] ("generic array creation"; 绕(T[])new Object[n]有警告) X instanceof List<String> Xstatic T字段。
- Math/包装: Math.max/min/abs/pow(b,e)/sqrt; Math.random()[0,1) → (int)(Math.random()*n) 0.n-1; Integer.MAX_VALUE; String.valueOf(42+"") → "42"; Number父类: x.doubleValue() 统一转double求和。

15 通配符?-PECS

- 数组协变OK,泛型不变: List<Person>=new ArrayList<Employee>() ×双向都不行: 要混装多种子类 → 声明成公共父类型 List<Object/Number>。
- △错的时机不同: 泛型不变 → 编译错compile error(早发现,跑不起来); 数组协变存错型 → 运行时ArrayStoreException(Object[] a=new String[3]; a[0]=42×)。泛型把错提前到编译期=更安全。
- <?>=unknown type未知类型(选择题答案, ≠Object≠排除null); 读为Object, add仅null。

	读	写add	收哪些List
? extends P	当P OK	X(除null)	P及子类
? super B	仅Object	B及子类OK	B及父类

PECS: 读 → extends生产者; 写 → super消费者(能加=下界及其子类, 因每种父类盒子都装得下; 不能加父类)。错误全是编译错。Collections.sort(list, Comparator<? super T>) 即super实战。

判断(Fruit→ Apple, Orange): FB<Fruit>=new FB<Apple>() X编译错; FB<Apple>=new FB<Fruit>() X; FB<? super Apple>..setFruit(new Apple()) OK运行; FB<? extends Orange>..setFruit(..) X(extends禁用); raw FB b=new FB() → 编译+警告。方法签名 List<? extends Point> 才能收 List<LocationPoint>。

16 枚举Enum

```
enum Suit{
    Hearts(2, "Red"), Spades(3, "Black"); //常量最前, 分
    号结尾, 带参构造器
    private final int n; private final String
    colour;
    Suit(int n,String c){ this.n=n; colour=c; } //构
    造器必须private/默认
    public String getColour(){ return colour; } }
    Suit, Spades.ordinal()-1(声明序从0, 与字段值无关)
    name()-"Spades"
    values()全部数组(遍历) valueOf("Hearts") ==equals/
    值均可比
    enum Light{ Red{ public Light next(){return
    Green; } },
        Green{ public Light next(){return Yellow; } },
        Yellow{ public Light next(){return Red; } };
        public abstract Light next(); } //末尾必须声明, 否
    则编译错
```

- 不能new(常量=预创建固定实例); 不能extends类(隐式继承Enum) 不能被继承; 可implements接口; 可有字段/方法/重载/抽象方法(每常量各实现=状态机mf)。
- name()固定不可重写; toString()可重写(显示); 存取数据用name()配valueOf。枚举构造器public → 编译错(考过F)。

17 递归base+recursive-memo

```
int fact(int n){ if(n<=1) return 1; return
n*fact(n-1); } //f(4)=24
int fib(int n){ if(n<2) return n; return
fib(n-1)+fib(n-2); } //f0 1 1 2 3 5
int countLeaf() //树: 无孩子=叶(Review
真题)
    if(children.size()==0) return 1; //base
    int c=0;
    for(FamilyMember f:children) c+=f.countLeaf(); /
    /累加, 别for里return!
    return c; }
long countMessage(int k){ //卫星
M(k)=M(k-2)+M(k-3) (Review真题)
    if(k<2) return 0;
    else if(k==2||k==3) return 1;
    else return
countMessage(k-2)+countMessage(k-3); }
int sumDigits(int n){ if(n==0) return 0; return
n%10+sumDigits(n/10); }
```

- base case(终止return)+recursive(参数向base收敛); 缺/不收敛 → StackOverflowError; 每调用→栈帧(参数+局部变量+返回地址)内存存大。
- 调用前print=正序(3,2,1下行); 调用后print=逆序(1,2,3回升); base停哪决定次数(停n==1比n==0少一层: g(n)=g(n-1)*2, g(1)=1 → g(4)=8非16)。传对象=传引用, 递归中改对象全程可见。
- memo: if (cache.containsKey(n)) return cache.get(n); 查缓存存在递归调用前(命中即返); 算完cache.put → 指数降线性。

!输出题: if(n/2==0)打"P" else打"O" m(n-1) → m(4): OOOO(整除! 1/2=0)。手算: 一路递归到base再倒着代回。

18 JUnit-assert-找错真题

```
import static org.junit.Assert.*; import
org.junit.*; //必背两行
public class CalcTest{
    Calc c;
    @Before public void setUp(){ c=new Calc(); } //
    每个测试前(隔离)
    @Test public void t1(){
    { assertEquals(15, c.add(5,10)); }
    @Test(expected=IllegalArgumentException.class) /
    /期望抛异常
        public void t2(){ c.setAge(-5); } }
    javac -cp .:junit-4.12.jar:hamcrest-core-1.3.jar
    CalcTest.java
    java -cp .:junit-4.12.jar:hamcrest-core-1.3.jar
    org.junit.runner.JUnit4 CalcTest
    • 断言: assertEquals(expected, actual) 用equals/True/False/Null/NotNull/assertSame(==同对象)/NotEquals/Hamcrest.assertThat(acts, is(exp)) : @After每测试后, @Before/Class类级一次; 黑盒=输入输出, 白盒=看内部(单元测试); 策略Positive/Negative/Edge。
    • jar=打包的库; -cp=classpath去哪找类; :=当前目录; :分隔(Win用); 无空格; 运行器JUnitCore, 测试类单独文件XxxTest.java, 同包不import被测类, 跨包/JUnit要import。
```

- assert 布尔; 假 → AssertionError(Error); 默认关 java -ea开; 别做参数校验(用IllegalArgumentException)。测异常也可try{...fail();}catch(X e){查消息}。

找错真题5错: 接口带实现要default; 接口继承接口=extends; 类实现接口=implements; 构造器无void; 要return非println; 调方法带括号b.capacity()。

19 匿名类- Lambda-Comparator-Q18

```
SayHello a=new SayHello(){ //匿名类
    public void hello(){ System.out.println("Hi"); }
}; //!结尾分号
SayHello b=(-)>System.out.println("Hi"); //lambda
0p add(x,y) → -x+y; Func f=x->x*2; //单参值()
(a,b) → { int s=a+b; return s; } //多行{须
return
// *各种排序(Collections.sort就地排, void; list.sort(c)
同效)
Collections.sort(nums); //Integer/String无
comparator=默认升序(自然序)
sort(nums, (a,b) → Integer.compare(a,b)); //int升
序 / 降序对调(b,a)
sort(ws, (a,b) → a.compareTo(b)); //String
升序 / 降序b.compareTo(a)
sort(ws, (a,b) → a.length()-b.length()); //按长度
sort(people, (a,b) →
>Integer.compare(a.getAge(), b.getAge())); //按
int字段
sort(people, (a,b) →
>Double.compare(a.getGpa(), b.getGpa())); //
double必用Double.compare! 别相减
sort(people, (a,b) →
>a.getName().compareTo(b.getName())); //按
String字段
sort(people, (a,b) → { int
s=Integer.compare(a.getAge(), b.getAge()); //多级: 先
年龄
        return s!=0 ? s :
a.getName().compareTo(b.getName()); }); //同岁再
比名字
Collections.max(list); Collections.reverse(list);
• 匿名类4规则: 1. 基于已有接口/类(不能凭空) 2. 无构造器(没名字; 初始化用裸()块) 3. 外部局部变量须effectively final 4. 一次1类型, 生成Outer$1.class。用变量名.方法()调用。
• lambda只用函数式接口(恰1抽象方法: default/static/Object不算 → Comparator方法多但只1抽象所以能用); 多方法接口只能匿名类。@FunctionalInterface防加第二个; this: 匿名类=自己, lambda=外围类; lambda不生成.class; 方法引用System.out::println。
• compare(a,b)返回: <0(负) → a排前 / ==0 → 相等 / >0(正) → a排后(升序逻辑: a比b小就返负把a放前); String: compareTo(Integer.compare; double: Double.compare; 减去精度X); 升序(a,b)降序(b,a)对调; 多级: 主键s!=0直接return s, 否则次键c; Collections.sort(list, c) 就地void; list.sort同; Comparable=类内compareTo自然序, Comparator=外部规则。Collections.max/reverse。
```

```
List<Covid19> list=new ArrayList<>(); //Q18: 读
文件→升序打印国名
Scanner sc=new Scanner(new
File("CovidCases.txt"));
while(sc.hasNextLine()){ String
line=sc.nextLine().trim(); //国名含空格(New
Zealand)→逐行读+切最后空格
    int i=line.lastIndexOf("
");
        list.add(new
Covid19(line.substring(0,i), Integer.parseInt(line.
substring(i+1)))); }
Collections.sort(list, (a,b) →
>Integer.compare(a.getCases(), b.getCases())); //
降序(b,a)
for(Covid19 c: list)
    System.out.println(c.getCountryName());
//等价匿名类: new Comparator<Covid19>(){public int
compare(Covid19 a, Covid19 b){return a.getCases()-
b.getCases();}}
```

20 Javadoc-JDB

/** */- javadoc生成HTML; 标签@param @return @throws @see @since @deprecated; 命令javadoc -d docs *.java。JDB调试: javac -g -jdb M - stop at M:32断点 → run → locals/print x/step单步(逻辑错编译器查不出用调试器)。

21 找错必查表(Q16)-速记

- main(String[] args) []: args用 .length非size(); String元素 → parseInt才能比/赋int。
- for(int i; ...) 未初始化; =vs==; String内容用equals非==。
- 非void漏return; void却return值X; 调方法漏括号b.capacityX。
- abstract类/接口被new; 子类没实现全部abstract; 枚举构造器public X; 枚举new X。
- 接口implements接口X(extends); 类extends接口X(implements); 构造器写void → 变普通方法; 接口方法带实现没default。
- super(..)this(..)非首行; 父类只有带参子类缺super(args); checked没catch/throws; 父类引用调子类独有方法。
- 缺分号/括号不配/大小写(Public X); 泛型 List<Object>=List<String>X, new T()X, List<int>X; 1/2=0; 数组下标0..length-1越界=运行时; 对null调方法NPE; 局部变量未赋值就用。

速记: int=0=RunTime exception!= 引用equals内容[checked=File/IO/default=接口方法有实现|边界仅<T extends>=<?>=unknown type|接口可被extend T|对象可作参数T|数组协变泛型不变|enum不能new/extends|implements|接口static方法不被继承|lambda要函数式|接口assert默认关-ea|字段看声明型方法看对象|先后子后构造|String不可变|primitive不能null|定义类=新数据类型 T|Array/List非定长|LinkedList节点任意内存T|类多类重继承F接口可|throw动作throws声明|finally必执行|Error别catch